

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO1: *Analyse DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships. (BL1, BL2)*

Activity Tool : Concept Mapping (Medium)

Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Construct a concept map linking the following terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.	BL2 – Understand
2	Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.	BL2 – Understand
3	Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.	BL2 – Understand
4	Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.	BL2 – Understand
5	Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.	BL2 – Understand

Code of Conduct:

I Kadhirezhilan. D [192511228] certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

RUBRICS FOR CALCULATION:

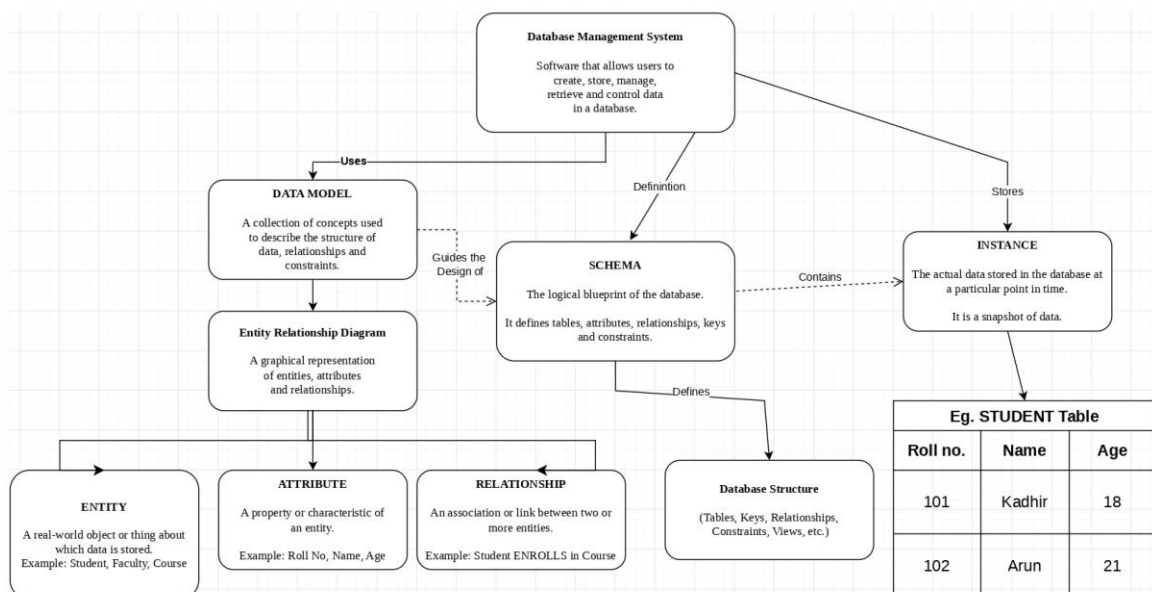
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Q1. Construct a Concept Map Linking the Following Terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.

Introduction

A **Database Management System (DBMS)** is software used to create, store, organize, retrieve, and manage data efficiently. According to the database design process, a **Data Model** provides the conceptual representation of data, which is converted into an **Entity-Relationship (ER) Diagram**. The ER Diagram consists of **Entities**, **Attributes**, and **Relationships**. After the database design is completed, the **Schema** defines the logical structure of the database, while the **Instance** represents the actual data stored at a particular time.

Concept Map



Concept Explanation

1. Database Management System (DBMS)

A DBMS is software that manages databases by allowing users to create, store, retrieve, update, and secure data.

Examples: MySQL, Oracle, SQL Server, PostgreSQL.

2. Data Model

A Data Model defines how data is organized, related, and stored within a database.

Common Data Models

- Hierarchical Model
- Network Model
- Relational Model
- Entity-Relationship Model

3. ER Diagram

An ER Diagram is a graphical model used during database design. It visually represents entities, their attributes, and the relationships among them.

4. Entity

An Entity is any real-world object or concept about which data is stored.

Examples

- Student
- Employee
- Department
- Course

5. Attribute

Attributes describe the characteristics of an entity.

Example

Student

- Student_ID
- Name
- Age
- Department

6. Relationship

A Relationship shows how entities are connected.

Examples

- Student **enrolls in** Course
- Employee **works in** Department
- Customer **places** Order

7. Schema

A Schema is the logical blueprint of the database that defines tables, attributes, keys, relationships, and constraints.

Important Point

- Changes very rarely.
- Defines the overall database structure.

8. Instance

An Instance is the actual content of the database at a specific moment.

Example:

Student_ID Name Department

101 Arun CSE

102 Priya AI & DS

The instance changes whenever records are inserted, updated, or deleted.

Relationship Among the Concepts

- **DBMS** uses a **Data Model** to organize and manage data.
- The **Data Model** is represented using an **ER Diagram**.
- The **ER Diagram** contains **Entities**, **Attributes**, and **Relationships**.
- **Entities** represent real-world objects.
- **Attributes** describe the properties of entities.
- **Relationships** connect entities.
- The **Schema** defines the logical structure of the database.
- The **Instance** contains the actual data stored in the database.
- The **Schema** remains relatively constant, whereas the **Instance** changes whenever data is modified.

Real-World Example

Consider a **College Database**:

- **DBMS:** MySQL
- **Data Model:** Relational Model
- **ER Diagram:** Student — Enrolls — Course
- **Entity:** Student, Course
- **Attribute:** Student_ID, Name, Course_ID
- **Relationship:** Student enrolls in Course
- **Schema:** Defines the Student and Course tables with keys and constraints.
- **Instance:** The current records of students and courses stored in the database.

Conclusion

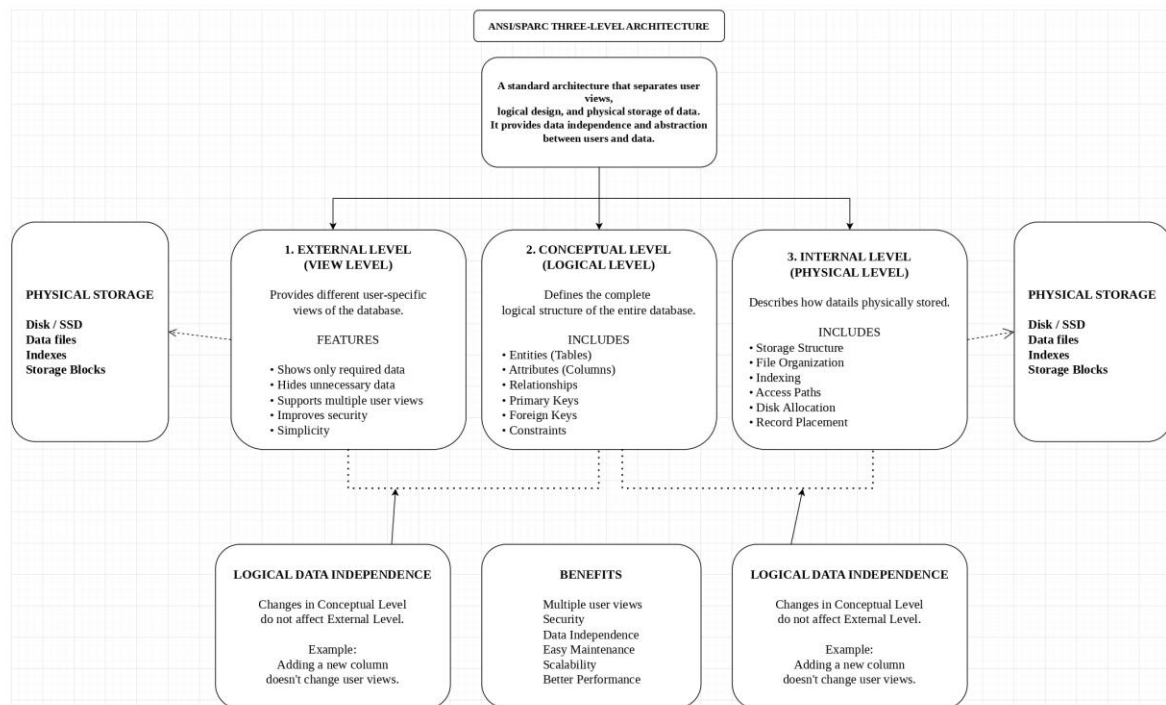
The **DBMS** serves as the central system for managing databases. It uses a **Data Model** to organize data and an **ER Diagram** to design the database by identifying **Entities**, **Attributes**, and **Relationships**. The **Schema** defines the permanent logical structure of the database, while the **Instance** represents the current data stored in it. Together, these concepts provide the foundation for designing efficient, accurate, and reliable database systems.

Q2. Create a Concept Map for the Three-Level ANSI/SPARC Architecture Showing Data Independence at Each Level

Introduction

The **ANSI/SPARC Three-Level Architecture** is a standard database architecture proposed to separate user applications from the physical storage of data. It consists of three levels: **External Level**, **Conceptual Level**, and **Internal Level**. This architecture provides **Logical Data Independence** and **Physical Data Independence**, allowing changes at one level without affecting the other levels. It improves flexibility, security, maintainability, and performance in database systems.

Concept Map



Explanation of the Concepts

1. ANSI/SPARC Architecture

The ANSI/SPARC architecture is a three-level database architecture that separates user views, logical database design, and physical data storage. This separation provides flexibility and supports data independence.

2. External Level (View Level)

The External Level is the highest level of the architecture. It provides customized views of the database to different users based on their requirements.

Features

- Displays only required information.
- Hides confidential data.
- Supports multiple user views.
- Improves database security.

Example

- Student → Views Marks
- Faculty → Updates Attendance
- Accountant → Views Fee Details

3. Conceptual Level (Logical Level)

The Conceptual Level represents the complete logical structure of the database. It defines all entities, attributes, relationships, constraints, and keys without considering physical storage.

Includes

- Tables
- Attributes
- Primary Keys
- Foreign Keys
- Relationships
- Constraints

Example

A college database contains Student, Faculty, Course, Department, and Attendance tables with relationships among them.

4. Internal Level (Physical Level)

The Internal Level describes how data is physically stored inside the computer system.

Includes

- Storage Blocks
- Files
- Indexes
- Access Methods
- Disk Organization
- Record Storage

Its main objective is to improve storage efficiency and database performance.

Data Independence

Data Independence is the ability to modify one level of the database architecture without affecting the higher levels.

Logical Data Independence

Logical Data Independence means changes made at the **Conceptual Level** do not affect the **External Level**.

Example

A new column **Email** is added to the Student table.

Students who only view marks continue using the same application without any modifications.

Physical Data Independence

Physical Data Independence means changes made at the **Internal Level** do not affect the **Conceptual Level**.

Example

The Database Administrator creates indexes, compresses files, or moves data to another storage device.

The database structure and application programs remain unchanged.

Relationship Between the Three Levels

- The **External Level** provides different database views for different users.
- The **Conceptual Level** defines the complete logical database structure.
- The **Internal Level** stores the data physically on storage devices.
- The **Conceptual Level** connects the External and Internal Levels.
- **Logical Data Independence** protects user views from logical changes.
- **Physical Data Independence** protects the logical database from physical storage changes.

Real-World Example (College Database Management System)

External Level

- Student → Views Marks
- Faculty → Enters Attendance
- Administrator → Manages Student Records

Conceptual Level

Defines the complete database structure:

- Student Table
- Faculty Table
- Course Table
- Department Table
- Attendance Table
- Relationships among all tables

Internal Level

Stores the database using:

- SSD/Hard Disk
- Data Files
- Indexes
- Storage Blocks
- File Organization Techniques

Advantages of ANSI/SPARC Architecture

- Provides multiple customized user views.
- Improves database security.
- Supports Logical Data Independence.
- Supports Physical Data Independence.
- Simplifies database maintenance.
- Reduces application development effort.
- Improves scalability and performance.
- Makes database modifications easier without affecting users.

Conclusion

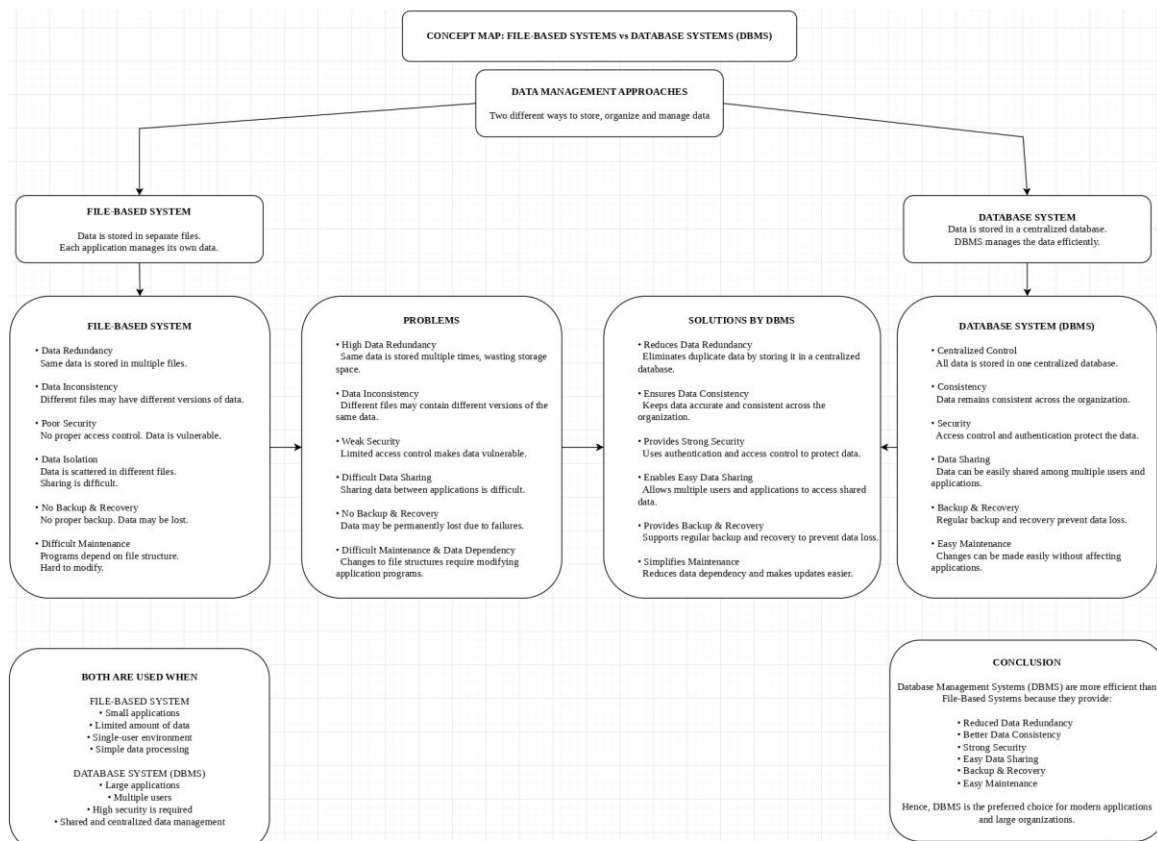
The **ANSI/SPARC Three-Level Architecture** is a fundamental database architecture that separates the database into **External**, **Conceptual**, and **Internal** levels. This separation ensures efficient database management while providing **Logical Data Independence** and **Physical Data Independence**. It improves security, flexibility, scalability, and maintainability, making it the foundation of modern Database Management Systems.

Q3. Map the Relationship Between File-Based Systems and Database Systems, Highlighting the Problems Solved by DBMS

Introduction

A **File-Based System** stores data in separate files managed by individual applications. A **Database Management System (DBMS)** stores and manages data in a centralized database. DBMS overcomes many limitations of file-based systems by providing efficient data management, security, consistency, and data sharing.

Concept Map



Comparison

File-Based System	Database Management System (DBMS)
Data stored in separate files	Data stored in a centralized database
High data redundancy	Reduces data redundancy
Data inconsistency	Maintains data consistency
Difficult data sharing	Easy data sharing
Weak security	Strong security with access control
No backup and recovery	Backup and recovery available
Limited concurrent access	Supports multiple users simultaneously
Difficult maintenance	Easy maintenance

Problems in File-Based System

- High data redundancy.
- Data inconsistency.
- Data isolation.
- Difficult data sharing.
- Weak security.
- No backup and recovery.
- Data dependency.
- Difficult maintenance.

Solutions Provided by DBMS

- Minimizes data redundancy.
- Maintains data consistency.
- Enables centralized data sharing.
- Provides authentication and authorization.
- Supports backup and recovery.
- Allows concurrent access by multiple users.
- Maintains data integrity using constraints.
- Simplifies database maintenance.

Real-World Example

College Management System

File-Based System

- Student, Faculty, and Fee records are stored in separate files.
- Duplicate data is common.
- Updating one file may not update others.

DBMS

- All records are stored in one centralized database.
- Data is shared among departments.
- Updates are reflected throughout the system.
- Security and backup are maintained.

Conclusion

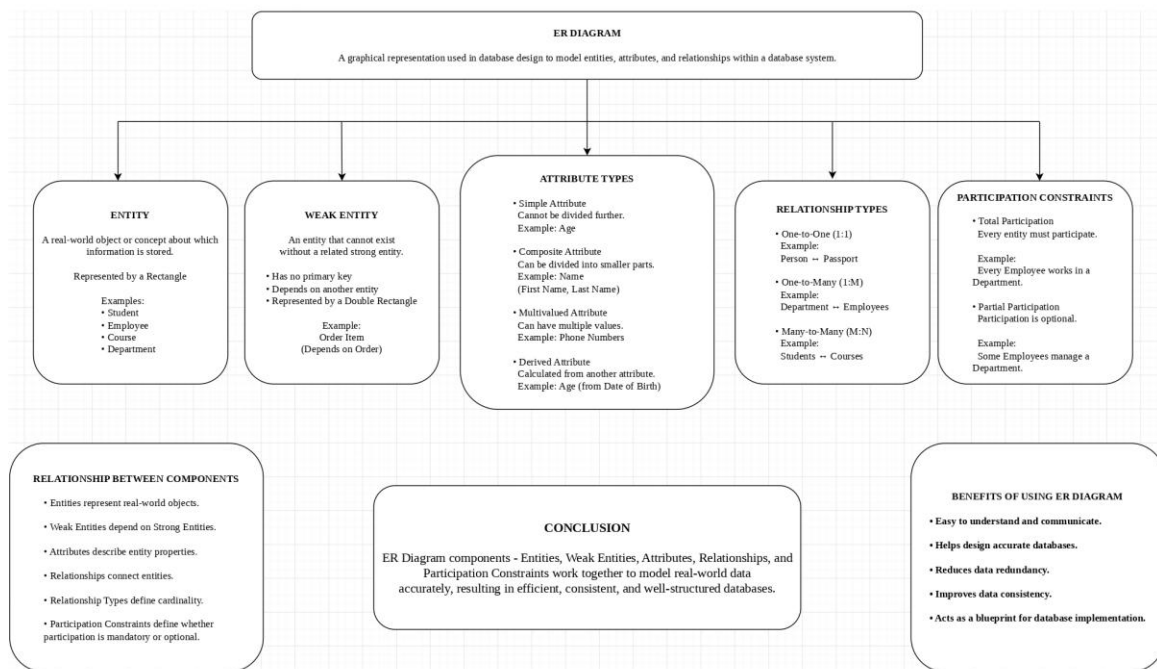
A **File-Based System** is suitable only for small applications but suffers from redundancy, inconsistency, and poor security. A **DBMS** provides centralized storage, reduces redundancy, ensures consistency, improves security, supports multiple users, and simplifies database management, making it the preferred solution for modern applications.

Q4. Design a Concept Map for ER Diagram Components: Entity, Weak Entity, Attribute Types, Relationship Types, and Participation Constraints

Introduction

An **Entity-Relationship (ER) Diagram** is a graphical representation of a database used during the design phase. It illustrates **entities, weak entities, attributes, relationships, and participation constraints**, helping designers create an accurate and efficient database structure.

Concept Map



Explanation of Components

1. Entity

An **Entity** is a real-world object or concept about which information is stored.

Examples

- Student
- Employee
- Course
- Department

2. Weak Entity

A **Weak Entity** cannot exist without a related strong entity. It does not have its own primary key and depends on another entity for identification.

Example

- Employee → Dependent
- Order → Order Item

3. Attribute Types

Attributes describe the properties of an entity.

Simple Attribute

Cannot be divided further.

Example: Age

Composite Attribute

Can be divided into smaller parts.

Example: Name → First Name, Last Name

Multivalued Attribute

Can have multiple values.

Example: Phone Numbers

Derived Attribute

Calculated from another attribute.

Example: Age derived from Date of Birth

4. Relationship Types

Relationships describe how entities are associated.

One-to-One (1:1)

One entity is related to only one entity.

Example: Person ↔ Passport

One-to-Many (1:M)

One entity is related to many entities.

Example: Department → Employees

Many-to-Many (M:N)

Many entities are related to many entities.

Example: Students ↔ Courses

5. Participation Constraints

Participation constraints specify whether every entity must participate in a relationship.

Total Participation

Every entity must participate.

Example: Every Employee works in a Department.

Partial Participation

Participation is optional.

Example: Some Employees manage a Department.

Relationship Between Components

- An **ER Diagram** represents the database structure.
- It contains **Entities** and **Weak Entities**.
- Every **Entity** has one or more **Attributes**.
- Attributes may be **Simple, Composite, Multivalued, or Derived**.
- **Relationships** connect entities.
- Relationship types include **One-to-One, One-to-Many, and Many-to-Many**.
- **Participation Constraints** define whether participation is **Total** or **Partial**.

Real-World Example (College Database)

- **Entity:** Student, Course
- **Weak Entity:** Student Fee Payment
- **Simple Attribute:** Student_ID
- **Composite Attribute:** Student Name
- **Multivalued Attribute:** Phone Number
- **Derived Attribute:** Age
- **Relationship:** Student enrolls in Course (Many-to-Many)
- **Participation:** Every Student must enroll in at least one Course (Total Participation).

Conclusion

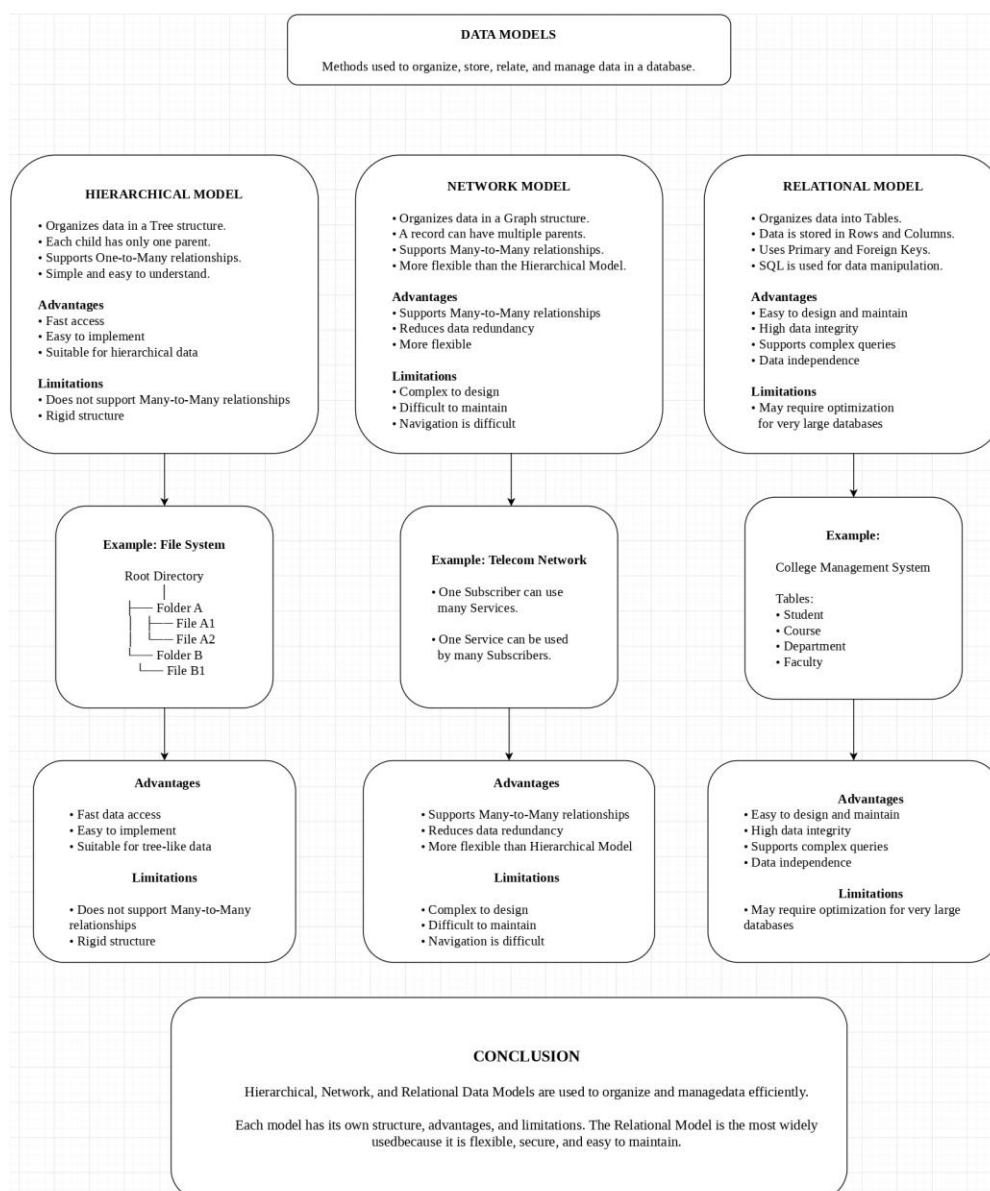
The **ER Diagram** is an essential tool for database design. It organizes data using **Entities, Weak Entities, Attributes, Relationships, and Participation Constraints**, ensuring a clear, consistent, and efficient database structure. Understanding these components helps in designing reliable and scalable database systems.

Q5. Construct a Concept Map Comparing Hierarchical, Network, and Relational Data Models with Their Key Properties

Introduction

A **Data Model** defines how data is organized, stored, and related in a database. The three major database models are **Hierarchical**, **Network**, and **Relational**. Each model has a different structure, relationship type, advantages, and applications.

Concept Map



Comparison of Data Models

Feature	Hierarchical Model	Network Model	Relational Model
Structure	Tree	Graph	Table
Relationship	One-to-Many	Many-to-Many	Keys (Primary & Foreign)
Data Organization	Parent–Child	Linked Records	Rows and Columns
Flexibility	Low	Medium	High
Complexity	Simple	Complex	Easy to Use
Query Language	Navigation	Navigation	SQL
Example	File System	Telecom Network	Banking System

Explanation of Data Models

1. Hierarchical Data Model

The Hierarchical Model organizes data in a **tree structure**, where every child record has only one parent.

Characteristics

- Tree structure
- Parent–Child relationship
- One-to-Many relationship
- Fast data access
- Simple design

Advantages

- Fast retrieval
- Easy implementation
- Suitable for hierarchical data

Limitation

- Does not support Many-to-Many relationships.

2. Network Data Model

The Network Model organizes data using a **graph structure**, allowing one record to have multiple parent and child records.

Characteristics

- Graph structure
- Many-to-Many relationships
- Flexible connections
- Linked records

Advantages

- Reduces data redundancy
- Supports complex relationships
- Faster navigation than hierarchical model

Limitation

- Complex database design and maintenance.

3. Relational Data Model

The Relational Model stores data in **tables (relations)** consisting of rows and columns.

Relationships are established using **Primary Keys** and **Foreign Keys**.

Characteristics

- Table structure
- SQL support
- High flexibility
- Data integrity
- Normalization

Advantages

- Easy to design and maintain
- Supports complex queries
- Strong security and integrity
- Most widely used database model

Limitation

- May require optimization for very large databases.

Relationship Between the Models

- All three are **Data Models** used in DBMS.
- **Hierarchical Model** stores data in a tree structure.
- **Network Model** stores data in a graph structure.
- **Relational Model** stores data in tables.
- The **Relational Model** is more flexible because it supports SQL, normalization, and key-based relationships.
- The **Network Model** is more flexible than the Hierarchical Model but more complex to maintain.

Real-World Applications

Hierarchical Model

- Organization Charts
- File Directory Systems
- XML Documents

Network Model

- Airline Reservation Systems
- Telecom Networks
- Social Network Connections

Relational Model

- Banking Systems
- College Management Systems
- Hospital Management Systems
- E-commerce Applications

Conclusion

The **Hierarchical**, **Network**, and **Relational** data models are fundamental database models used for organizing data. The **Hierarchical Model** is suitable for simple parent-child relationships, the **Network Model** supports complex many-to-many relationships, and the **Relational Model** provides flexible, secure, and efficient data management using tables and SQL. Due to its simplicity and scalability, the **Relational Model** is the most widely used model in modern Database Management Systems.